

Cours 20 — Trier et fusionner des tables

Semaine 20 ■ Manuel Hachette ch. 7 (§ 7.4) ■ 1 h

Objectifs

- Trier une table suivant une colonne avec `sorted` et une fonction clé
- Fusionner deux tables sur un identifiant commun

L'autre créneau d'1 h de la semaine est consacré au **DS n° 3** (dichotomie, tris, tables).

1. Trier une table suivant une colonne

Nos tris (semaines 16-17) comparent des nombres. Pour trier des **enregistrements**, il faut dire *selon quoi* : une **fonction clé** qui extrait la valeur de comparaison.

```
def cle_population(e):
    """Clé de tri : la population de l'enregistrement."""
    return e["population"]

table_triee = sorted(table, key=cle_population)
```

- `sorted` renvoie une **nouvelle** liste triée (la table d'origine est intacte); `table.sort(key=...)` trie en place.
- Ordre décroissant : `reverse=True`.
- On passe la fonction **sans parenthèses** — comme les gestionnaires d'événements de la semaine 7.

Maintenant qu'on a écrit deux tris nous-mêmes, on a le droit d'utiliser celui de Python : on sait ce qu'il fait, et ce qu'il coûte (un $O(n \log n)$, meilleur que nos n^2).

Tri à plusieurs critères : renvoyer un **tuple** — Python compare composante par composante :

```
def cle_dep_puis_nom(e):
    """Clé : département, puis nom en cas d'égalité."""
    return (e["departement"], e["nom"])
```

2. Fusionner deux tables

Deux fichiers open data : les communes (code INSEE, nom, population) et les résultats d'un recensement (code INSEE, logements). Pour croiser : la **fusion** (jointure) sur l'**identifiant commun** — le code INSEE, sans doublons (semaine 19!).

```
def fusion(t1, t2, cle):
    """Renvoie la table fusionnant t1 et t2 sur la clé commune.

    Préconditions : cle sans doublons dans t2 ;
    tout enregistrement de t1 a un correspondant dans t2.
    """
    # index : valeur de clé -> enregistrement de t2
    index = {}
    for e in t2:
        index[e[cle]] = e
```

```
resultat = []
for e in t1:
    correspondant = index[e[cle]]
    f = dict(e) # copie (semaine 10 !)
    for champ in correspondant:
        f[champ] = correspondant[champ]
    resultat.append(f)
return resultat
```

L'**index** (un dictionnaire) évite de rechercher le correspondant par un parcours de **t2** à chaque enregistrement de **t1** : accès direct par clé, fusion en un passage par table au lieu de $n_1 \times n_2$ comparaisons.

Que faire d'un enregistrement **sans correspondant** ? L'ignorer, le garder incomplet, ou signaler l'erreur : un choix à documenter — c'est la partie délicate des données réelles (TP du jour).

À retenir

- `sorted(table, key=fonction)` — la fonction clé extrait la valeur de comparaison ; `reverse=True` pour décroissant.
- Multi-critères : la clé renvoie un tuple.
- Fusion sur identifiant commun sans doublons ; un `index` (dictionnaire) rend la fusion linéaire.
- Toujours décider et documenter le sort des enregistrements sans correspondant.